Контролируемое исследование размерности состояния для недельного управления кредитными порогами

Содержание

1 Введение **3**

2 Что именно изучается и почему это важно **4**
2.1 Почему исследуется именно недельное управление порогами 4
2.2 Почему отложенные исходы здесь принципиальны 4
2.3 Почему важно разделение на новых и повторных клиентов 5

3 Граница между reference-проектом и текущей реализацией **5**

4 Как устроен симулятор **6**
4.1 Временная структура эпизода 6
4.2 Что происходит внутри одной недели 6
4.3 Уточнение по метрике `expected_profit` 7

5 Сценарии и их исследовательский смысл **7**

6 Контролируемый эксперимент по размерности состояния **8**
6.1 Что зафиксировано 8
6.2 Что меняется 8
6.3 Как устроены слои состояния 9
6.4 Почему дополнительные признаки вообще могут помочь 9

7 Контроллеры **9**
7.1 Базовые политики 9
7.2 RL-алгоритмы 10

8 Функция награды и смысл метрик **10**
8.1 Как устроена награда 10
8.2 Почему “лучшая по прибыли” и “лучшая по reward” политика могут не совпадать 11
8.3 Какие метрики используются 11

9 Экспериментальный протокол **11**

10 Основные результаты **12**
10.1 Лучший overall-контроллер . . . 12
10.2 Лучший RL-контроллер . . . 13
10.3 Разные RL-алгоритмы реагируют на размерность по-разному . . . 13

11 Кросс-размерные графики и что они показывают **15**

12 Сценарная интерпретация результатов **17**
12.1 Почему одной общей таблицы недостаточно . . . 17
12.2 Смысловая карта сценариев в 50D . . . 18
12.3 Adverse stress . . . 18
12.4 Base market . . . 18
12.5 Class-imbalance shift . . . 19
12.6 Drift . . . 19
12.7 Noise . . . 19
12.8 Split-policy dynamics . . . 19

13 Что показывают отдельные рисунки по размерностям **20**

14 Поведение порогов и важная интерпретационная деталь **21**
14.1 Почему нулевая threshold volatility не означает полное отсутствие различий . 21
14.2 Какие пороговые пары выбирают лучшие RL-контроллеры . . . 21
14.3 Почему это все равно важный результат . . . 21

15 Обсуждение **22**
15.1 Почему 30D выглядит наиболее сбалансированной размерностью . . . 22
15.2 Почему 50D выглядит как saturation, а не как новый скачок . . . 22
15.3 Почему baseline по-прежнему выигрывает overall . . . 22
15.4 Практический смысл trade-offs . . . 22

16 Ограничения исследования **23**

17 Научная новизна и вклад **23**

18 Заключение **24**

1 Введение

Настоящий текст описывает актуальное состояние репозитория после того, как он был фактически перестроен вокруг контролируемого эксперимента по размерности состояния в задаче недельного управления порогами одобрения в кредитном скоринге. Это важно подчеркнуть сразу: исследование не сводится к обычной задаче бинарной классификации заявок и не сравнивает модели на статическом табличном датасете в стиле “одобрить или отклонить отдельный объект”. В центре внимания находится динамическая портфельная политика. Контроллер раз в неделю выбирает пороги для новых и повторных клиентов, а затем именно эти недельные пороги определяют состав принятой когорты, будущие дефолты, возвраты, поздние recovery-потоки и, в конечном счете, финансовое качество всего портфеля.

Такая постановка делает исследование содержательно более близким к реальной практике управления кредитным потоком. В операционном процессе компания редко вручную пересматривает решение по каждой отдельной заявке как независимой единице. Намного типичнее ситуация, когда на определенный период времени задается или корректируется скоринговый cutoff, после чего бизнес наблюдает, как изменение этого cutoff влияет на объем выдач, качество новых когорт, нагрузку на капитал и последующие денежные потоки. Именно поэтому в репозитории объектом управления является не точечное решение по заемщику, а недельная пороговая политика.

Вторая принципиальная особенность исследования состоит в том, что финансовый результат здесь является *отложенным*. Если кредит был выдан сегодня, его качество не раскрывается немедленно. Часть кредитов погашается позже, часть дефолтит, а часть дефолтов частично восстанавливается еще через несколько недель. Следовательно, нельзя оценивать политику по мгновенной выгоде текущей недели. Политика может выглядеть привлекательной локально, но оказаться слабой на полной траектории портфеля. И наоборот, более строгий режим одобрения может временно выглядеть хуже, зато на горизонте всей симуляции дать более чистый и устойчивый результат.

Текущий эксперимент в репозитории задает очень конкретный исследовательский вопрос: *как меняется качество RL-контроллера, если увеличивать размер наблюдения с 12 до 20, 30 и 50 признаков, не меняя при этом остальные элементы постановки?* Такой вопрос ценен именно своей контролируемостью. Если меняется только `state_dim`, то различия в результатах можно интерпретировать как различия в информативности и сложности пространства состояний, а не как следствие смены среды, набора сценариев, функции награды или конкурентных baseline-политик.

Поэтому главный вывод исследования должен формулироваться аккуратно. Репозиторий не показывает, что reinforcement learning уже однозначно превосходит все rule-based подходы. Наоборот, лучший overall-контроллер в quick-профиле остается не-RL baseline. Но репозиторий очень убедительно показывает другое: дизайн состояния существенно влияет на качество RL, рост размерности полезен лишь до определенного уровня, а польза от richer state сильно зависит от конкретного алгоритма.

2 Что именно изучается и почему это важно

2.1 Почему исследуется именно недельное управление порогами

Если смотреть на кредитный скоринг только как на задачу предсказания дефолта, то возникает соблазн считать, что все сводится к хорошей модели PD и выбору одного фиксированного threshold. Но в реальной кредитной политике этого недостаточно. Бизнес интересует не только точность ранжирования клиентов, но и то, как выбранная политика меняет будущую структуру портфеля. Важно не просто "кого одобрить сегодня", а "какую недельную границу риска выдержит портфель так, чтобы одновременно не разрушить прибыльность, не разогнать дефолт и не перегрузить капитал".

Именно поэтому в проекте действие контроллера определено как недельная пара порогов для новых и повторных клиентов. После выбора этих порогов симулятор генерирует поток заявок, применяет пороги, формирует принятую когорту, а далее разворачивает ее кредитную судьбу во времени. Такое представление делает задачу последовательной, стратегической и существенно более интересной, чем статическая классификация.

С исследовательской точки зрения это означает следующее. Во-первых, задача действительно становится reinforcement learning problem, потому что текущее решение влияет на будущее состояние портфеля. Во-вторых, качество политики можно обсуждать на языке, который близок к предметной области: прибыль, дефолтность, approval rate, capital usage, устойчивость награды, реакция на shift-сценарии.

2.2 Почему отложенные исходы здесь принципиальны

В обычной статической формулировке кредитный риск часто мыслится как функция от признаков клиента в момент принятия решения. В данной работе этого недостаточно, потому что исследование моделирует весь путь кредита после одобрения. Выданный займ не приносит окончательный финансовый результат сразу. Он живет несколько недель или месяцев, может привести к положительному cash flow, отрицательному default-loss, а затем, возможно, к позднему recovery.

Именно эта отложенность и делает задачу содержательной. Если бы награда зависела только от ожидаемой выгоды текущей когорты, контроллер можно было бы оценивать почти миопически. Но в текущей реализации награда с delayed reward опирается на *реализованные* потоки, а не только на текущие ожидания. Это приводит к естественной исследовательской дилемме: политика, которая выглядит сильной "здесь и сейчас", может создать плохой хвост убытков позже; политика, которая в первые недели отстает, может оказаться лучше на полном горизонте.

В репозитории этот смысл подтверждается не только кодом, но и экспортированным диагностическим артефактом `outputs/dim_30/locally_worse_globally_better.csv`. Он показывает, что в adverse-stress среде некоторые агенты сначала проигрывают локально, а затем сильно выигрывают на финальной траектории. Например, для 30D DQN ранний разрыв зафиксирован примерно как -643.46, а финальный разрыв как 250,578.20. Это не означает, что RL обязательно сильнее в stress-сценариях. Это означает другое: ранняя и итоговая оценка политики могут принципиально расходиться, а значит delayed-outcome логика является не декоративной, а методологически необходимой.

2.3 Почему важно разделение на новых и повторных клиентов

В среде новые и повторные заемщики не являются одним и тем же сегментом с переименованными колонками. Для них различаются параметры распределения скоринговых баллов, чувствительность default probability к score, вероятность recovery, размеры займа и сроки кредита. Это экономически реалистично: повторные клиенты обычно содержат дополнительный behavioral signal и нередко имеют другую риск-доходность, чем новые.

Из этого следует простой, но важный вывод: единый shared threshold может быть заведомо неэффективен. Поэтому split-policy логика здесь не второстепенная опция, а часть основной постановки. Контроллер в нормальном режиме выбирает не один порог, а пару порогов. Более того, один из сценариев специально построен так, чтобы новые клиенты ухудшались, а повторные, наоборот, улучшались. В такой среде полезность отдельного контроля по сегментам становится практически прямым предметом эксперимента.

Это особенно важно для темы размерности состояния. Рост числа признаков нужен не сам по себе. Он имеет смысл лишь тогда, когда добавляемые координаты помогают лучше описывать ту сложность, которая реально есть в задаче. В нашем случае такой сложностью являются сегментная неоднородность, история портфеля, лаги по прибыли и дефолту, gap между new и repeat thresholds, динамика капитальной нагрузки и т.д.

3 Граница между reference-проектом и текущей реализацией

Согласно `notes/reference_study.md`, текущий проект сохраняет несколько ключевых идей исходной reference-работы: недельное взаимодействие, пороговое управление, delayed outcomes, delayed rewards, warm-up период и проверку устойчивости в shift-сценариях. Однако актуальный репозиторий нельзя описывать как простое воспроизведение старой схемы.

В текущей реализации появились важные расширения.

- Split-policy контроль для новых и повторных клиентов стал дефолтной логикой, а не частным случаем.
- Эксперименты стали config-driven: параметры профиля, горизонта, bootstrap CI и набора контроллеров задаются через текущие YAML-конфиги.
- Введен контролируемый эксперимент по размерности состояния, в котором сравниваются 12D, 20D, 30D и 50D наблюдения.
- Функция награды стала явно constraint-aware: она учитывает default-rate, approval-rate, capital-usage и volatility penalties.
- Набор RL-алгоритмов включает DQN, Double-DQN, PPO и SAC, а набор baseline-методов покрывает статические, миопические, реактивные и constraint-aware стратегии.
- В репозитории появились writer-handoff материалы, figure map, table map, экспорт CI-кривых и сводные кросс-размерные таблицы.

Для текста диплома это означает, что нельзя опираться на старую интуицию о “простом одномерном state” или “едином пороге”. Источник истины теперь находится в текущем коде, конфигурации и экспортированных результатах.

4 Как устроен симулятор

4.1 Временная структура эпизода

В `src/rl_credit_scoring_sim/env/simulator.py` эпизод разбит на три логические фазы.

1. **Warm-up.** В течение 8 недель симулятор работает на дефолтных порогах, чтобы сформировать начальный пул незавершенных кредитов, rolling-метрики прибыли и исторические показатели дефолтов.

2. **Interactive horizon.** Затем начинаются 26 интерактивных недель quick-профиля, в которых контроллер реально управляет порогами.

3. **Terminal settlement.** После завершения интерактивного горизонта все оставшиеся interactive-cashflow события закрываются, чтобы эпизод не обрывался искусственно на середине жизненного цикла недавно выданных кредитов.

Последний пункт особенно важен. Без terminal settlement политика, которая в конце эпизода активно выдает длинные кредиты, систематически выглядела бы хуже, чем есть на самом деле, потому что положительные потоки по этим кредитам просто не успели бы попасть в measured horizon. Текущая реализация корректирует эту проблему.

4.2 Что происходит внутри одной недели

Каждая неделя начинается с формирования scenario-specific market state. Сценарий задает динамику доли repeat-клиентов, тренд объемов, сезонность, сдвиги score distributions, сдвиги default pressure, уровень шума, коэффициент размера займа, качество recovery и параметры shock-эпизода.

После этого симулятор генерирует weekly application counts отдельно для новых и повторных клиентов, рисует score distributions, строит default probabilities через сегментные sigmoid-функции, определяет will-default / will-recover траектории и задает loan amount и duration. Далее выбранные пороги применяются к обоим сегментам. Все заявки с score выше соответствующего порога принимаются.

Именно на этом месте проявляется главная экономическая идея исследования. Одобренные кредиты не сразу превращаются в итоговую прибыль. Они лишь создают будущие события: repayment, default-loss и, при необходимости, late recovery. Поэтому в текущей неделе одновременно существуют два разных типа информации.

- Характеристики *текущей принятой когорты*: `expected_profit_current`, `expected_npv_current`, `expected_default_rate_current`.

- *Реализованные* потоки по ранее выданным кредитам: `realized_profit`, `realized_npv`, realized default counts и т.д.

Такое разделение делает симулятор методологически сильнее. Он не смешивает сегодняшние ожидания и сегодняшние реализованные потоки, а позволяет анализировать, как текущее решение будет раскрывать себя во времени.

4.3 Уточнение по метрике `expected_profit`

Важная деталь, которую нужно зафиксировать в тексте диплома: экспортированные CSV-файлы сохраняют название `expected_profit_mean`, но в текущем коде агрегированная episode-level метрика вычисляется как сумма *реализованной* прибыли по интерактивным неделям плюс terminal settlement. Иными словами, это legacy-name, а не буквальное "математическое ожидание прибыли" в узком смысле.

Поэтому в интерпретации результатов удобно делать так: при ссылке на конкретные CSV можно сохранять repository terminology и говорить `expected_profit_mean`, но содержательно пояснять, что это итоговая реализованная прибыль портфеля внутри эпизода. Такое уточнение согласует текст с реальным кодом `episode_summary()` и убирает возможную терминологическую путаницу.

5 Сценарии и их исследовательский смысл

Шесть активных сценариев описаны в `configs/scenarios.yaml`. Они нужны не просто для визуального разнообразия. Каждый сценарий проверяет определенный механизм управления.

Сценарий	Что именно он проверяет
`base_market`	Стабильный рынок без сильной структурной поломки. Это лучший сценарий для проверки, умеет ли контроллер использовать информативное состояние без борьбы с резким regime shift.
`drift`	Медленное ухудшение качества, прежде всего у новых клиентов, плюс рост объема. Проверяется способность уловить постепенную деградацию и стать более селективным до того, как портфель существенно испортится.
`adverse_stress`	Резкий негативный shock, более высокие дефолты, шум и повышенный pressure на капитал. Здесь наказываются слишком агрессивные политики.
`class_imbalance_shift`	Ускоренный рост доли более слабого сегмента новых клиентов. Сценарий показывает, насколько политика умеет работать не только с общим approval rate, но и с composition risk.
`noise`	Более высокий measurement noise и нестабильные weekly volumes. Это тест на устойчивость к шуму и случайным колебаниям входного потока.
`split_policy_dynamics`	Новые клиенты ухудшаются, а повторные улучшаются. Это самый прямой тест полезности split thresholds и сегментно-специфического состояния.

Таблица 1: Логика сценариев из `configs/scenarios.yaml`.

Именно наличие этих сценариев позволяет осмысленно обсуждать, *почему* один и тот же контроллер выигрывает в одних режимах и проигрывает в других. Без сценарной структуры весь анализ свелся бы к безликой средней метрике, что сильно обеднило бы научную интерпретацию.

6 Контролируемый эксперимент по размерности состояния

6.1 Что зафиксировано

Сильная сторона текущего репозитория состоит в том, что эксперимент действительно сделан controlled. Во всех четырех запусках постоянными остаются:

- среда, сценарии и недельная логика симуляции;
- split-policy action semantics;
- диапазон порогов от 35 до 85 с шагом 5;
- delayed reward, risk-aware shaping и penalty logic;
- warm-up, interactive horizon и terminal settlement;
- seeds 11, 23, 37;
- train/eval protocol и набор контроллеров;
- bootstrap-CI логика и формат экспортируемых артефактов.

Это делает эксперимент содержательно чистым. Когда меняется только `state_dim`, можно честно говорить об эффекте размерности состояния, а не о сумме несогласованных изменений.

6.2 Что меняется

Единственный экспериментальный фактор:

$$\texttt{state_dim} \in \{12, 20, 30, 50\}.$$

Причем рост размерности сделан кумулятивно: 20D содержит весь 12D baseline и добавляет новый слой, 30D содержит 20D и добавляет исторические признаки, 50D содержит 30D и добавляет более богатые сегментные и stability-признаки.

6.3 Как устроены слои состояния

Размер	Содержательная интерпретация
12D	Компактная недельная панель: где находится эпизод, какой был approval rate, как выглядит rolling realized default, какова текущая profit/volatility картина, каков capital pressure и какие thresholds применялись.
20D	12D плюс немедленный экономический контекст: expected profit и expected NPV на заявку, realized NPV, weekly reward, repeat-share, threshold gap и headroom по капиталу.
30D	20D плюс память о недавней истории: лаги approval/default/profit/capital usage, rolling averages, rolling std по прибыли и lagged threshold deltas.
50D	30D плюс более детализированное сегментное и stability-описание: segment-specific approval/default/profit признаки, доли принятых новых и повторных клиентов, rolling reward moments, cumulative profit/reward to date, volatility капитальной нагрузки и история threshold-gap.

Таблица 2: Смысл слоев состояния. Точный порядок признаков и их определения даны в `state_dimension_manifest.md`.

Очень важно, что в коде есть отдельная programmatic check, подтверждающая неизменность первых 12 признаков между 12D, 20D, 30D и 50D. Для научной добросовестности это принципиально: эксперимент не должен “тихо” менять baseline-ядро состояния, пока заявляет, что сравнивает только расширения.

6.4 Почему дополнительные признаки вообще могут помочь

Растущая размерность в этой работе не является бессодержательным увеличением числа координат. Добавляемые признаки имеют конкретный экономический смысл.

- В 20D контроллер начинает видеть не только факт изменения метрик, но и непосредственный экономический контекст последней недели.
- В 30D у него появляется историческая память, позволяющая отличать шум от устойчивого изменения режима.
- В 50D состояние становится более детализированным в сегментном и stability-смысле, но при этом усложняется обучение.

Именно поэтому гипотеза исследования формулируется так: дополнительный state signal должен быть полезен до некоторого уровня, после чего начнут проявляться diminishing returns или even optimization burden. Экспортированные результаты подтверждают именно такую форму зависимости.

7 Контроллеры

7.1 Базовые политики

Baseline-методы в проекте хорошо подобраны для содержательного сравнения.

- `static_threshold` — полностью фиксированный общий порог.
- `split_policy_static` — фиксированная, но уже сегментно-разделенная политика.
- `profit_oriented` — еженедельный поиск по сетке, максимизирующий текущую cohort-level прибыль.
- `risk_aware_weekly` — hand-crafted feedback policy, которая меняет пороги по rolling realized default, approval pressure и capital usage.
- `constraint_aware_weekly` — weekly grid search с явным учетом penalty-weighted нарушений ограничений.

Это важно, потому что исследование не сравнивает RL с заведомо слабыми оппонентами. Оно сравнивает его с несколькими разумными управленческими логиками: статической, split-static, миопической profit-search, реактивной risk-aware и явно constraint-aware.

7.2 RL-алгоритмы

В репозитории реализованы `dqn`, `double_dqn`, `ppo` и `sac`. DQN и Double-DQN естественно подходят для дискретной сетки порогов. PPO и SAC работают в непрерывном пространстве действий, которое затем маппится обратно на тот же диапазон порогов и ту же гранулярность. Это делает сравнение честным: RL-методы различаются способом оптимизации политики, но не тем, какие конечные пороги среда в принципе допускает.

Для интерпретации это важно по двум причинам. Во-первых, можно различить эффект state dimensionality и effect of action representation. Во-вторых, становится видно, что richer state далеко не одинаково полезен для разных классов алгоритмов.

8 Функция награды и смысл метрик

8.1 Как устроена награда

В `src/rl_credit_scoring_sim/env/reward.py` награда собирается из нескольких частей. При включенном delayed reward основной сигнал строится на realized profit и realized NPV, а сверху накладываются risk-aware shaping и penalty-компоненты за default-rate, approval-rate, capital-usage и volatility.

В упрощенной записи можно сказать, что weekly reward имеет вид

$$
\begin{aligned}
r_t = {} & 1.0 \cdot \Pi_t^{realized} + 0.25 \cdot \mathrm{NPV}_t^{realized} - 0.35 \cdot \hat{d}_t \cdot 100 \\
& - 9.0 \cdot \left[\max(0, \bar{d}_t - 0.12)\right]^2 \cdot 1000 \\
& - 2.0 \cdot |\alpha_t - 0.42| \cdot 1000 \\
& - 3.5 \cdot \left[\max(0, c_t - 0.82)\right]^2 \cdot 1000 \\
& - 1.4 \cdot \max\left(0, \frac{\sigma_{\Pi,t} - 8500}{8500}\right) \cdot 1000.
\end{aligned}
\tag{1}
$$

Для смысловой интерпретации это означает, что политика оценивается не только по грубой “денежной выгоде”, но и по тому, какой ценой эта выгода достигается. Можно заработать

больше, но сделать это при избыточном approval pressure, более высокой realized default dynamics или сильной перегрузке капитала. В таком случае shaped reward справедливо скорректирует рейтинг.

8.2 Почему “лучшая по прибыли” и “лучшая по reward” политика могут не совпадать

Это не теоретическая возможность, а реально наблюдаемая особенность текущих результатов. В `noise` сценарии highest-profit controller — это `risk_aware_weekly`, который достигает примерно 97,377.95 по метрике `expected_profit_mean`. Но reward-based winner там остается `profit_oriented`, у которого прибыль ниже — около 94,589.22, зато итоговый cumulative reward выше при действующих штрафах и весах.

Этот пример особенно полезен для объяснения комитету. Он показывает, что исследование изучает именно управленческий компромисс, а не просто “кто принес больше денег любой ценой”. Для кредитной политики это концептуально более адекватно.

8.3 Какие метрики используются

Основные метрики результатов:

- approval rate;
- default rate;
- итоговая портфельная прибыль, экспортированная под legacy-именем `expected_profit_mean`;
- realized portfolio NPV;
- cumulative shaped reward;
- средняя projected capital usage ratio;
- reward volatility;
- stability index;
- threshold volatility.

Здесь нужно сделать еще одно важное уточнение. `threshold_volatility` в коде — это стандартное отклонение *изменений* порога между соседними неделями внутри эпизода. Значит, нулевая volatility не означает, что разные seeds выбирают одинаковые пороги; она означает, что *внутри данного эпизода* порог по неделям не двигался.

9 Экспериментальный протокол

Основной evidence set соответствует профилю `quick`. В merged configuration это означает:

- 8 warm-up недель;

- 26 интерактивных недель;
- 260 заявок в неделю до применения scale factors;
- `train_scale` = 0.80, `validation_scale` = 0.90, `test_scale` = 1.00, при этом `test_subset_fraction` = 0.65;
- 12 training episodes на RL seed;
- 4 evaluation runs на seed и сценарий;
- seeds 11, 23, 37;
- 95% bootstrap CI с 120 resamples в активном fast-CI режиме.

Результаты складываются в `outputs/dim_12/`, `outputs/dim_20/`, `outputs/dim_30/`, `outputs/dim_50` а кросс-размерные summary files — в корень `outputs/`. Это очень удобно для дипломного текста, потому что можно обсуждать и отдельные размерности, и сводную картину по всем размерностям одновременно.

Отдельного внимания заслуживает validation layer. Код dimensionality pipeline проверяет неизменность первых 12 признаков, согласованность протокола, отсутствие future leakage в added features, полноту прогонов для каждого контроллера и внутреннюю согласованность cross-dimension tables. Для академической работы это важный аргумент в пользу надежности результатов: observed saturation effect не является побочным следствием скрытой несогласованности эксперимента.

10 Основные результаты

10.1 Лучший overall-контроллер

Размер	Лучший overall	Type	Profit	Reward	Default rate
12	`profit_oriented`	baseline	76 022.08	88 080.60	0.1169
20	`profit_oriented`	baseline	76 022.08	88 080.60	0.1169
30	`profit_oriented`	baseline	76 022.08	88 080.60	0.1169
50	`profit_oriented`	baseline	76 022.08	88 080.60	0.1169

Таблица 3: Лучший overall-контроллер по каждой размерности из `outputs/overall_best_by_dimension.csv`.

На первый взгляд это выглядит как простая и даже немного разочаровывающая картина: лучший контроллер не меняется и во всех четырех случаях остается baseline. Но на самом деле этот результат очень информативен.

Во-первых, он подтверждает чистоту controlled design. Baseline-политики не зависят от увеличения state vector, и потому их положение действительно должно оставаться тем же самым. Если бы лучший overall controller неожиданно резко менялся только из-за роста `state_dim`, это было бы подозрительно.

Во-вторых, важно не преувеличивать доминирование `profit_oriented`. Второй baseline, `constraint_aware_weekly`, имеет cumulative reward 88,000.75, то есть отстает всего примерно на 79.85. Это означает, что вершина overall-ranking занята не “чудо-победителем”, а двумя очень сильными rule-based стратегиями, одна из которых более aggressively profit-seeking, а другая почти столь же сильна, но более constraint-aware.

10.2 Лучший RL-контроллер

Размер	Лучший RL	Profit	NPV	Reward	Approval rate	Capital usage
12	`dqn`	59 197.47	57 144.61	67 308.36	0.4159	0.4454
20	`double_dqn`	61 096.44	58 978.98	69 812.02	0.4329	0.4569
30	`dqn`	66 758.29	64 755.07	78 219.69	0.3527	0.3902
50	`dqn`	66 946.81	64 815.05	78 198.39	0.3935	0.4280

Таблица 4: Лучший RL-контроллер по каждой размерности из `outputs/best_rl_by_dimension.csv`.

Именно здесь и находится главный эмпирический результат исследования.

Переход 12D → 20D дает умеренное улучшение. Переход 20D → 30D дает самый сильный прирост. А переход 30D → 50D почти не приносит дополнительного cumulative reward. Это и есть observed saturation.

Особенно важно, что 30D улучшает качество RL не через грубое расширение approval volume. Наоборот, для лучшего RL-контроллера 30D approval rate снижается до 0.3527, а capital usage — до 0.3902. И при этом reward и profit заметно растут. Это очень сильный исследовательский сигнал: дополнительная историческая информация помогает не просто “выдать больше”, а сформировать *качественно более удачный портфель*.

50D в этом смысле выглядит уже не как качественный новый скачок, а как режим diminishing returns. Profit растет лишь на примерно 188.53, cumulative reward даже слегка уменьшается, а approval rate и capital usage снова повышаются. То есть богатое состояние все еще может быть полезно, но новая сложность уже почти не окупается.

10.3 Разные RL-алгоритмы реагируют на размерность по-разному

Это один из самых важных выводов, который отсутствовал в старом коротком тексте. Нельзя говорить о влиянии размерности только на уровне “best RL frontier”, потому что разные алгоритмы реагируют принципиально по-разному.

Агент	Размер	Profit	Reward	Default rate	Threshold volatility
dqn	12	59 197.47	67 308.36	0.1233	0.0000
dqn	20	-15 413.93	-34 719.92	0.1545	0.0000
dqn	30	66 758.29	78 219.69	0.1185	0.0000
dqn	50	66 946.81	78 198.39	0.1195	0.0000
double_dqn	12	59 221.98	66 915.54	0.1223	0.0000
double_dqn	20	61 096.44	69 812.02	0.1260	0.0000
double_dqn	30	57 467.87	65 247.13	0.1282	0.0000
double_dqn	50	-49 782.91	-79 329.16	0.1574	0.0000
ppo	12–50	12 196.74	4 958.59	0.1537	0.0000
sac	12	-36 379.87	-57 961.65	0.1674	3.0132
sac	20	-22 232.63	-35 997.61	0.1494	2.2454
sac	30	-11 904.25	-26 218.31	0.1535	2.6183
sac	50	-72 433.15	-105 961.13	0.1677	1.9269

Таблица 5: Все RL-агенты по размерностям на основе per-dimension `main_overall_summary.csv`.

Из этой таблицы следуют очень содержательные выводы.

- DQN чувствителен к качеству representation: он хорош на 12D, проваливается на 20D, затем резко улучшается на 30D и удерживает это качество на 50D.
- Double-DQN достигает пика раньше: лучший результат у него на 20D, затем идет ухудшение, а на 50D наблюдается уже явный collapse.
- PPO в текущем quick-профиле практически идентичен по всем размерностям. Это означает, что увеличение state vector under current budget не меняет итоговую policy quality сколько-нибудь существенно.
- SAC остается отрицательным по reward во всех размерностях и особенно слабеет на 50D.

Для дипломной интерпретации это очень важный момент. Размерность состояния влияет не “вообще”, а через связку “размерность + конкретный способ оптимизации”. И именно поэтому вывод исследования должен быть аккуратным: richer state действительно способен помочь RL, но он не гарантирует автоматического улучшения для любого алгоритма.

11 Кросс-размерные графики и что они показывают

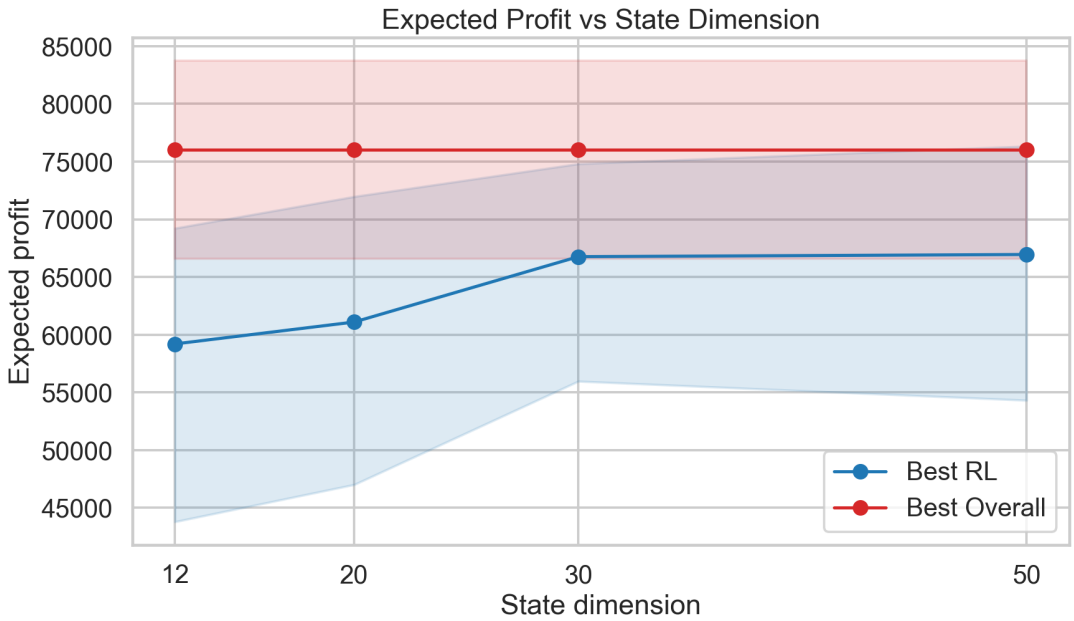


Рис. 1: Прибыль в зависимости от размерности состояния для best RL и best overall controller.

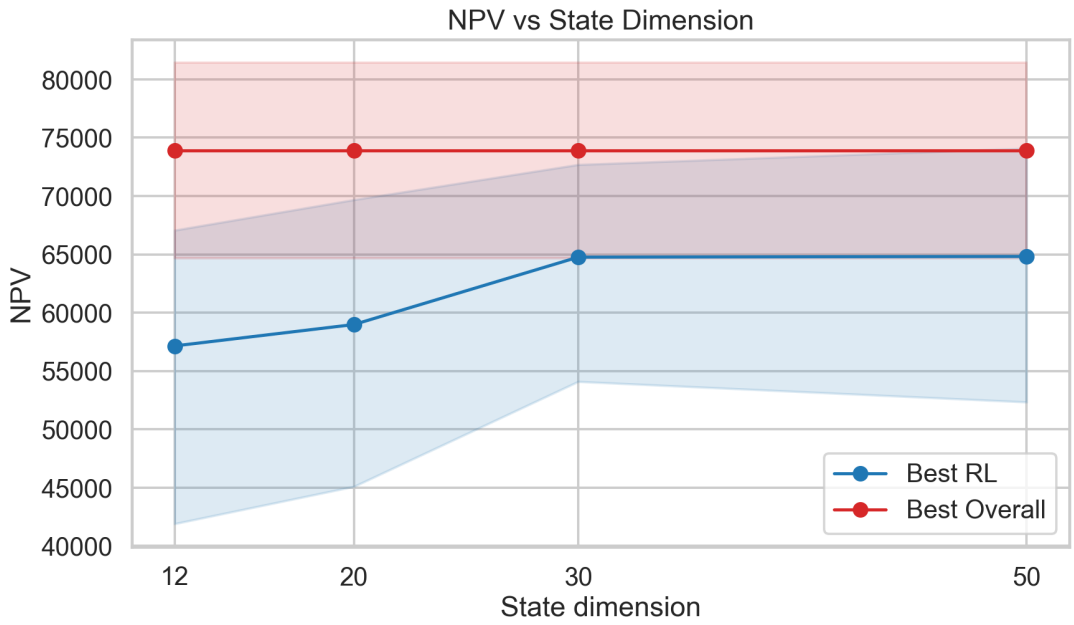


Рис. 2: NPV в зависимости от размерности состояния.

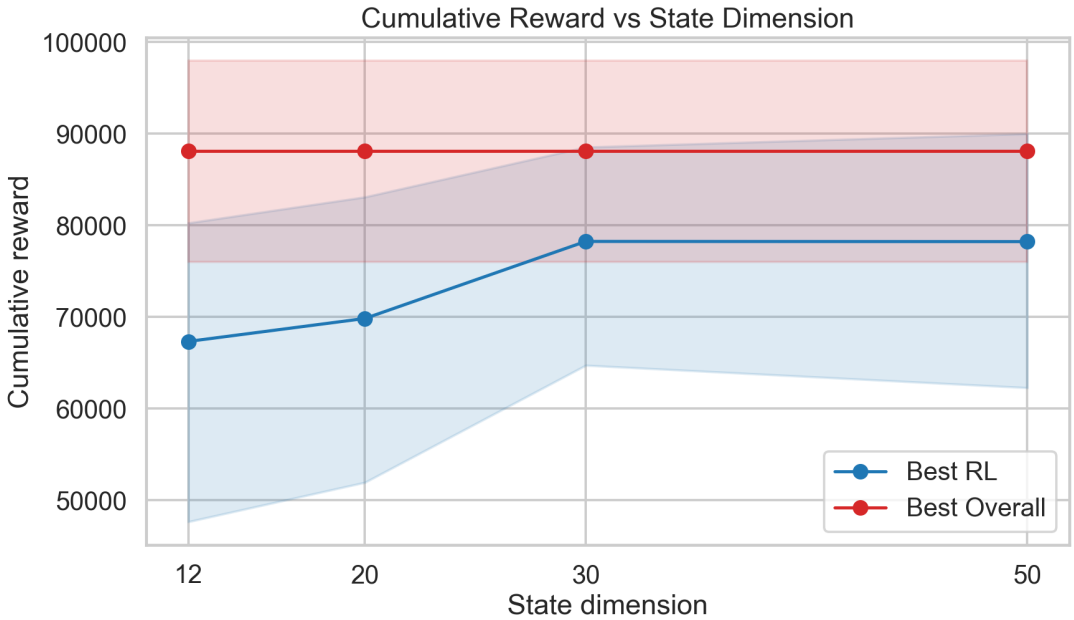


Рис. 3: Cumulative reward в зависимости от размерности состояния.

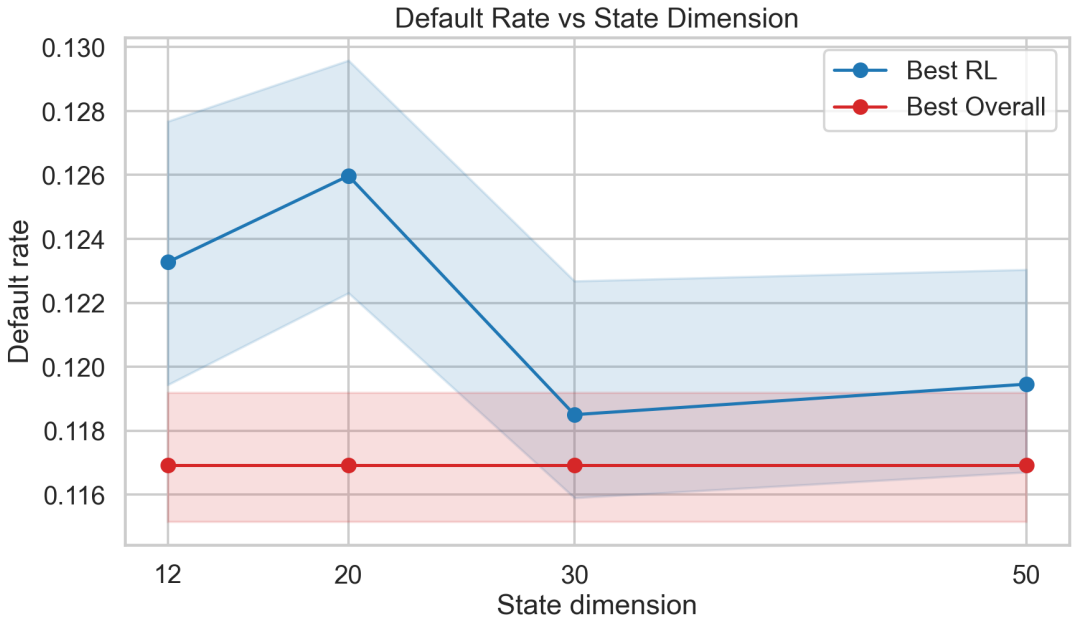


Рис. 4: Default rate в зависимости от размерности состояния.

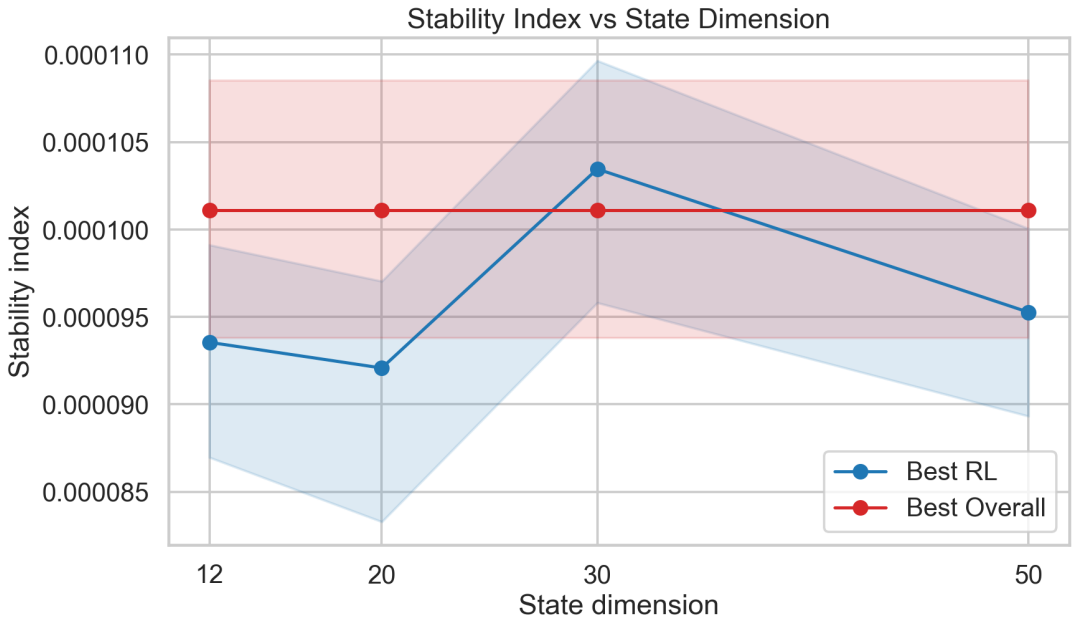


Рис. 5: Stability index в зависимости от размерности состояния.

Эти рисунки вместе очень наглядны. Плоская линия overall-best baseline означает, что comparison set действительно не меняется с ростом размерности. Растущая, а затем выравнивающаяся линия best RL означает, что additional state signal сначала помогает, а затем перестает заметно улучшать управление. Default-rate curve показывает, что ключевой выигрыш достигается к 30D. Stability-index figure показывает, что главное улучшение идет не через радикальный скачок устойчивости, а через лучшую комбинацию profit, selectivity и capital discipline.

12 Сценарная интерпретация результатов

12.1 Почему одной общей таблицы недостаточно

Overall-ranking полезен, но он скрывает важную структуру. Разные сценарии моделируют разные типы сложности: стабильный режим, медленный drift, сильный shock, composition shift, noise и diverging segment dynamics. Поэтому у исследования неизбежно возникает второй уровень анализа: нужно понять, в каких именно сценариях RL уже близок к baseline, в каких он действительно лучше, а в каких заметно уступает.

Кроме того, необходимо различать по крайней мере три понятия “лидера”:

- reward-based winner;
- highest-profit winner;
- lowest-default winner.

В shaped portfolio problem это не одно и то же.

12.2 Смысловая карта сценариев в 50D

Сценарий	Reward-based winner	Highest-profit winner	Lowest-default winner
`adverse_stress`	`profit_oriented`	`profit_oriented`	`profit_oriented`
`base_market`	`dqn`	`dqn`	`dqn`
`class_imbalance_shift`	`constraint_aware_weekly`	`constraint_aware_weekly`	`dqn`
`drift`	`profit_oriented`	`profit_oriented`	`profit_oriented`
`noise`	`profit_oriented`	`risk_aware_weekly`	`dqn`
`split_policy_dynamics`	`profit_oriented`	`profit_oriented`	`dqn`

Таблица 6: Различие между reward, profit и risk лидерами в 50D на основе `outputs/dim_50/main_scenario_summary.csv`.

Эта таблица очень полезна для объяснения результатов комиссии. Она показывает, что в исследовании нет одного универсального "победителя". В стабильном рынке RL действительно может оказаться лучшим и по reward, и по profit, и по default rate одновременно. В noise-сценарии highest-profit winner и reward-based winner расходятся. В class-imbalance-shift conservative baseline выигрывает по profit и reward, а DQN при этом дает наименьший default rate. Именно в таких случаях особенно хорошо виден содержательный trade-off, заложенный в дизайн исследования.

12.3 Adverse stress

Во всех размерностях adverse stress остается самой трудной средой для RL. Reward-based winner здесь всегда — `profit_oriented`, с прибылью около 29,336.42, cumulative reward около 25,150.74 и default rate около 0.1178. Лучшие RL-значения значительно хуже: в 12D best RL имеет отрицательную прибыль, в 30D DQN поднимается лишь до 3,413.22, а в 50D DQN снова уходит в отрицательную прибыль.

Такая картина логична. В adverse stress одновременно растут default pressure, noise и размер кредитной нагрузки. Цена ошибки становится очень высокой. Миопический на вид baseline `profit_oriented` здесь на самом деле быстро становится строгим, потому что уже текущая когорта выглядит существенно хуже. Он поджимает выдачу и за счет этого защищает портфель. RL в quick-profile partly учится той же логике, но в условиях короткого training budget не успевает consistently превратить richer state в столь же robust control.

12.4 Base market

Base market — это сценарий, в котором RL выглядит наиболее убедительно. Уже в 12D DQN выигрывает по reward. В 20D strongest RL становится Double-DQN с прибылью 94,321.51. В 50D DQN получает примерно 92,105.55 прибыли, 110,521.41 cumulative reward и default rate 0.1134, то есть оказывается лучше `profit_oriented` одновременно по нескольким ключевым метрикам.

Почему это правдоподобно? Потому что base market дает контроллеру наиболее "чистый" режим, где полезный сигнал не заслоняется сильной структурной поломкой. В такой среде

richer state действительно может позволить RL найти более тонкую границу селективности, чем rule-based weekly search, и извлечь выгоду из информационной структуры состояния.

12.5 Class-imbalance shift

В этом сценарии неизменно побеждает `constraint_aware_weekly`. Это очень содержательный результат. Сценарий специально наказывает политику, которая плохо реагирует на рост доли более слабого сегмента новых клиентов. Явно constraint-aware search здесь естественным образом оказывается очень сильным baseline.

Но одновременно видно, что richer-state DQN приближается к нему и по мере роста размерности сильно улучшает свой результат. В 50D DQN доходит примерно до 65,715.59 по прибыли и одновременно становится lowest-default winner с default rate около 0.1163. Это означает, что более богатое состояние помогает RL лучше видеть composition risk, хотя quick-profile все еще не дает ему выиграть по главной reward-based ranking.

12.6 Drift

Drift особенно важен для интерпретации 30D. Здесь `profit_oriented` остается reward-based winner, но best RL profit заметно растет от 12D к 30D и затем ослабевает на 50D. Такая форма полностью соответствует интуиции о значении исторических признаков: медленное ухудшение качества лучше всего ловится тогда, когда в состоянии появляется память о недавней динамике. Именно это и происходит на уровне 30D.

50D в этом сценарии уже не дает сопоставимого прироста. Значит, ключевой полезный signal для drift-среды уже содержится в историческом блоке 30D, а дальнейшее усложнение описания не увеличивает качество управления в той же мере.

12.7 Noise

Noise-сценарий особенно полезен для объяснения, что reward и profit — разные вещи. Highest-profit winner здесь — `risk_aware_weekly` с прибылью около 97,377.95. Но reward-based winner — `profit_oriented`. Одновременно 30D и 50D DQN показывают очень неплохую прибыль и при этом более низкий default rate, чем у risk-aware baseline.

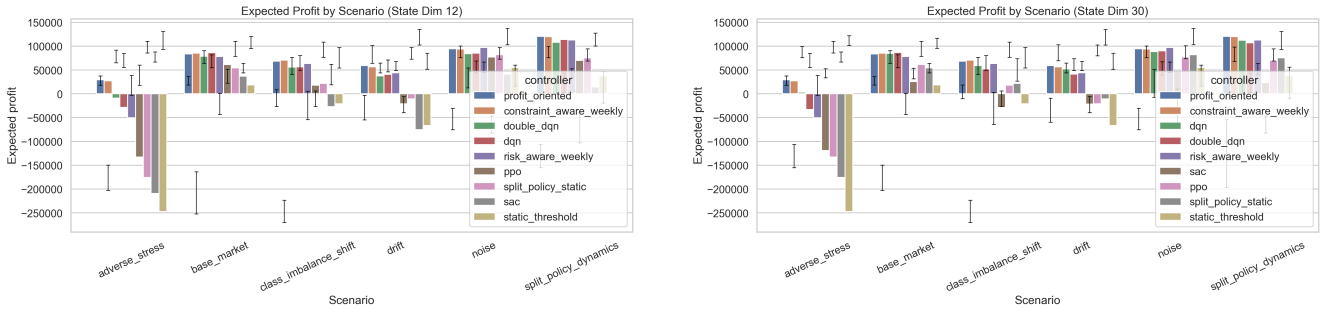
Это и есть практический смысл исследования: иногда richer-state RL не является лучшим по основной общей ranking, но создает более чистую кредитную политику, уменьшая дефолтность при вполне конкурентной прибыли. Для банковской интерпретации это очень существенный результат.

12.8 Split-policy dynamics

Этот сценарий буквально сконструирован для проверки split-policy logic. Reward-based winner остается `profit_oriented`, но с ростом размерности лучший RL-контроллер заметно приближается к нему. В 50D DQN получает примерно 119,918.57 прибыли против 120,324.99 у baseline, то есть gap становится очень маленьким. Более того, именно DQN в 50D показывает здесь наименьший default rate, около 0.1093.

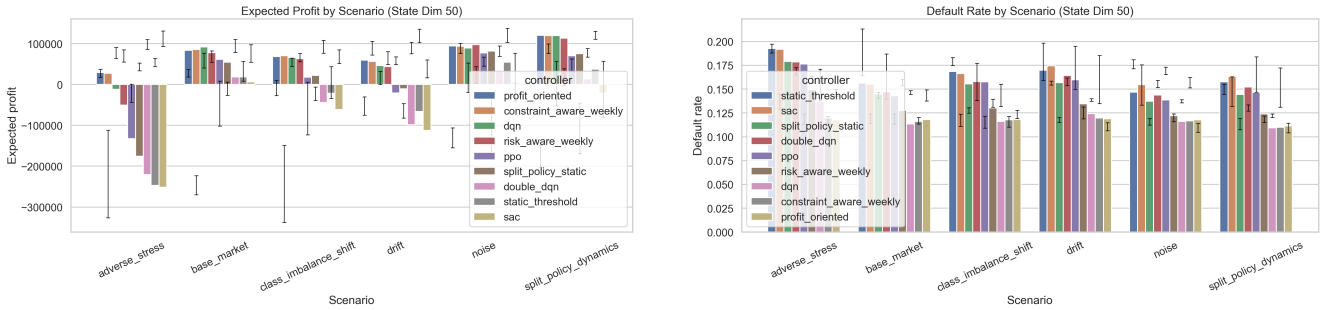
Это сильный содержательный аргумент в пользу richer segmented state. Когда динамика new и repeat сегментов расходится, более богатое состояние действительно помогает RL почти догнать profit-seeking baseline, а по качеству портфеля даже чуть превзойти его.

13 Что показывают отдельные рисунки по размерностям



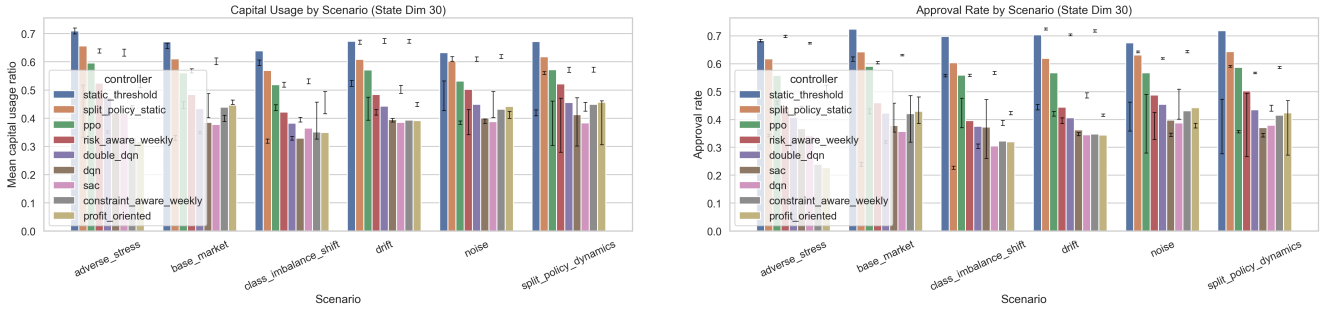
(a) 12D (b) 30D

Рис. 6: Scenario-level сравнение прибыли для двух характерных размерностей.



(a) 50D profit (b) 50D default rate

Рис. 7: Как богатое состояние меняет баланс между доходностью и риском.



(a) 30D capital usage (b) 30D approval rate

Рис. 8: Как 30D меняет селективность и загрузку капитала.

Эти рисунки полезны тем, что позволяют увидеть *структуру* улучшений. Например, 30D не просто “улучшает среднюю метрику”, а делает RL более селективным и более аккуратным

по capital usage. 50D, напротив, показывает, что дальнейший рост сложности уже не приводит к столь же качественному улучшению профиля политики.

14 Поведение порогов и важная интерпретационная деталь

14.1 Почему нулевая threshold volatility не означает полное отсутствие различий

В aggregated summary у лучших RL-строк `threshold_volatility_mean` равна нулю. Это не означает, что все RL-агенты выбирают один и тот же порог. Это означает, что внутри каждого эпизода порог по неделям не меняется.

Данные weekly logs подтверждают, что для best RL controller в каждой размерности 100% evaluation episodes являются constant-threshold episodes. Это очень важное наблюдение. В quick-profile richer state улучшает прежде всего *выбор удачной пары порогов на эпизод*, а не обучение по-настоящему "живой" weekly-adaptive траектории.

14.2 Какие пороговые пары выбирают лучшие RL-контроллеры

Экспорт weekly logs показывает следующие типичные пары.

- 12D DQN: $(65, 45)$, $(85, 35)$, $(85, 50)$.
- 20D Double-DQN: $(65, 40)$, $(75, 50)$, $(80, 55)$.
- 30D DQN: $(70, 65)$, $(85, 35)$, $(85, 50)$.
- 50D DQN: $(70, 45)$, $(85, 40)$, $(85, 45)$.

Это observation полезно для очень честной академической интерпретации. Текущие результаты сильнее подтверждают тезис "лучшее состояние помогает выбрать лучшую split-policy конфигурацию", чем тезис "лучшее состояние уже приводит к сложной адаптивной weekly-control траектории". Для quick-profile это важная граница корректного вывода.

14.3 Почему это все равно важный результат

Даже если best RL policy внутри эпизода почти не двигает thresholds, это не обесценивает исследование. Напротив, это показывает, что при текущем training budget основной выигрыш приходит от более качественного *policy selection*, а не от интенсивной week-to-week корректировки. Для диплома это полезно: можно честно показать и сильную сторону RL-state design, и текущую эмпирическую границу его адаптивности.

15 Обсуждение

15.1 Почему 30D выглядит наиболее сбалансированной размерностью

30D — это точка, в которой richer state еще добавляет значимый полезный signal, но еще не создает чрезмерной optimization complexity. Именно здесь наблюдается самый сильный скачок по лучшему RL reward, существенный рост прибыли и NPV, снижение default rate относительно 20D и одновременное сокращение approval rate и capital usage. Такой набор движений особенно важен: он показывает, что контроллер не "покупает" reward за счет более агрессивной выдачи, а улучшает качество отбора.

Содержательно это очень хорошо согласуется с природой added features. Между 20D и 30D в состояние добавляется историческая память: лаги по approval/default/profit/capital usage, rolling means, rolling std по прибыли и lagged threshold deltas. Для недельной портфельной задачи это, вероятно, наиболее ценный дополнительный блок.

15.2 Почему 50D выглядит как saturation, а не как новый скачок

50D добавляет сегментную детализацию, rolling reward features, cumulative-to-date metrics и историю threshold-gap. Эти признаки не бессмысленны. Но в текущем quick-profile они почти не двигают best RL frontier дальше. Более того, для части алгоритмов 50D оказывается даже вредным.

Это особенно видно по Double-DQN: он лучший RL на 20D, терпимый на 30D и катастрофически плохой на 50D. Следовательно, рост размерности здесь нужно понимать не как бесплатное обогащение информации, а как совместный рост и signal, и optimization burden.

15.3 Почему baseline по-прежнему выигрывает overall

Лидерство `profit_oriented` и близость `constraint_aware_weekly` вполне объяснимы. Эти политики напрямую используют структурную форму задачи.

- Они выполняют явный weekly search по той же сетке порогов, которую допускает среда.
- Им не нужно учить value function или policy approximation при коротком бюджете.
- Их логика напрямую связана с текущей экономикой когорты и с constraint-структурой reward.

Поэтому корректный вывод состоит не в том, что RL "проиграл", а в том, что в хорошо структурированном simulator strong rule-based policy search остается очень сильным benchmark. RL при этом становится существенно лучше, когда state design становится более информативным.

15.4 Практический смысл trade-offs

С прикладной точки зрения один из самых интересных моментов исследования — это то, как меняются approval rate, capital usage и default rate вместе с ростом размерности. 20D best

RL policy становится более одобряющей и более капиталоемкой. 30D best RL policy, наоборот, становится более селективной и одновременно более сильной по reward. 50D снова ослабляет эту дисциплину.

Для портфельного кредитного бизнеса это важнее, чем простое сравнение одной метрики. Смысл управления порогом заключается именно в том, чтобы найти workable compromise между объемом выдач, качеством портфеля, устойчивостью и использованием капитала.

16 Ограничения исследования

- Симулятор синтетический. Он экономически правдоподобен, но не является прямой калибровкой на production-данные банка.
- Quick-profile intentionally short. 12 training episodes на seed достаточно для controlled comparison, но недостаточно для сильных заявлений о полном convergence RL.
- Функция награды является спроектированной. Следовательно, ranking контроллеров зависит от выбранного баланса между доходностью и штрафами.
- Экспортированные CI отражают неопределенность внутри выбранного генератора сценариев, а не внешнюю валидность на реальном рынке.
- Лучшие RL-политики в текущем профиле в основном episode-static, поэтому evidence for truly dynamic weekly threshold adjustment пока слабее, чем evidence for better threshold-pair selection.

Эти ограничения не разрушают выводы, но задают корректную границу того, что именно уже показано, а что еще требует дальнейшей проверки.

17 Научная новизна и вклад

Новизна работы не в придумывании еще одного RL-алгоритма как такового, а в комбинации нескольких исследовательских идей в одной согласованной постановке.

- Кредитное управление формулируется как недельное портфельное threshold control, а не как однократная классификация.
- Delayed outcomes и terminal settlement встроены в саму среду, а не вынесены в постобработку.
- Новые и повторные клиенты моделируются раздельно и управляются через split-policy action.
- RL сравнивается не с примитивным baseline, а с набором экономически осмысленных rule-based контроллеров.
- Сам state design превращается в самостоятельный объект controlled empirical study.

Именно так и следует формулировать contribution диплома: это вклад в problem formulation, simulation design и controlled experimental methodology для RL-based credit policy research.

18 Заключение

Текущее состояние репозитория позволяет сделать четкий и хорошо аргументированный вывод. Увеличение размерности состояния с 12 до 20 и затем до 30 признаков существенно улучшает лучшую RL-политику. Наиболее сильный прирост достигается на 30D: растут прибыль, NPV и cumulative reward, одновременно улучшается default profile и снижается capital usage по сравнению с 20D winner. Переход 30D $\rightarrow$ 50D дает уже очень мало дополнительной пользы и показывает явный saturation effect.

При этом исследование не поддерживает упрощенный тезис “RL уже всегда лучше baseline”. Лучший overall-контроллер в quick-profile outputs — это по-прежнему `profit_oriented` baseline, а `constraint_aware_weekly` находится почти на том же уровне. Зато исследование очень убедительно показывает другое: если задача формулируется как weekly portfolio control с delayed outcomes, то качество RL сильно зависит от того, насколько содержательно сконструировано состояние.

Самый корректный финальный вывод звучит так. В рассматриваемом RL credit-scoring simulator richer state действительно создает полезный сигнал, но только до определенной точки. Исторический и портфельный контекст, добавленный к 30D, дает наибольший прирост качества. Дальнейшее расширение до 50D уже почти не улучшает лучший RL frontier и может ухудшать результат у отдельных алгоритмов. Следовательно, в текущей реализации именно дизайн состояния, а не только выбор алгоритма, является одним из ключевых факторов качества портфельного порогового управления.